PrepLang: a data-aware compiler for data-preparation pipelines that reuses its proofs on new batches

Nikhil V[a,*], Ranjithkumar S[b]

[a]*School of Computer Science and Engineering, Vellore Institute of Technology, Vellore 632014, Tamil Nadu, India*
[b]*Department of Analytics, School of Computer Science and Engineering, Vellore Institute of Technology, Vellore 632014, Tamil Nadu, India*

Abstract

In batch data preparation, a pipeline is compiled when its input already exists and is then reused on batches the compiler has never seen. PrepLang, a compiler written in C++17 for a small language of table and image pipelines, exploits both facts. At compile time it reads every CSV cell, the Parquet footers and the image headers, proves or refutes the pipeline's data contracts before any row is processed, and removes the checks and operations that this input makes redundant. Each removal is kept in the generated pandas and OpenCV program behind a guard holding the facts it relied on. On a new batch the program runs unchanged unless the batch breaks those facts; the removed code then runs again, and the result is the one a fresh compilation would give. Across 426 later months of five public datasets, the compiled pipelines agreed with a fresh compilation on 100% of the months and never produced a different result. The repository includes 306 test checks with sanitizer, coverage and fuzzing jobs, Lean 4 proofs for a core of the analysis, and one script that reproduces every number reported here.

Keywords: domain-specific language, data-aware compilation, abstract interpretation, speculative optimization, deoptimization, data validation

Nr.	Code metadata description	Please fill in this column
C1	Current code version	v2.1.0
C2	Permanent link to code/repository used for this code version	`https://github.com/Nikhi00718/preLang/tree/v2.1.0`
C3	Permanent link to Reproducible Capsule	Docker image `ghcr.io/nikhi00718/preplang:2.1.0` (`Dockerfile` in the repository); `eval/reproduce.sh check` builds, tests and regenerates every number of this article from the committed results
C4	Legal Code License	MIT
C5	Code versioning system used	git
C6	Software code languages, tools, and services used	C++17 (no external libraries); Python 3 with pandas, NumPy, PyArrow, OpenCV, matplotlib; Lean 4; CMake
C7	Compilation requirements, operating environments and dependencies	CMake 3.16+, a C++17 compiler (tested in CI: Visual Studio 2022 on Windows, GCC on Ubuntu and macOS); Python 3.12+ with pandas (2.0+, 3.0 tested), PyArrow, OpenCV, matplotlib (charts); Lean 4.34.1 (optional, for the proofs)
C8	If available, link to developer documentation/manual	`README.md` (user manual), `LANGUAGE.md` (reference), `ARCHITECTURE.md`, `CONTRIBUTING.md`; the READMEs of `eval/`, `paper/`, `proofs/`, `docs/`
C9	Support email for questions	atnikhil06@gmail.com

Table 1: Code metadata (mandatory)

*Corresponding author
Email addresses: `atnikhil06@gmail.com` (Nikhil V), `ranjithkumar.s@vit.ac.in` (Ranjithkumar S)

Table 2: PrepLang and the closest tools. *Values*: missing values and value ranges.

Tool	Decides at	Knows about the data	Follows code	On a new batch
TFDV [4], Deequ [5], GX [6], pandera [7]	run time	values, from each batch	no	validates it again
F# type providers [8]	compile time	types, from a sample	no	–
Zhuang and Lu [11]	compile time	column types, from the code	yes	–
Gerstl et al. [12]	compile time	R data-frame shapes, optionally from the data	yes	–
Mirji [13]	compile time	declared schemas	yes	checks the output schema
PrepLang	**compile time**	**types and values of every cell; image headers**	**yes**	**admits by certificate; reruns only the removals it breaks**

1. Motivation and significance

Much of the hidden cost of machine-learning systems sits in their data dependencies and glue code rather than in the models [1]. A large share of that glue is data preparation: a script over pandas [2] or OpenCV [3] that finds out only at run time whether the column it scales holds numbers, or whether the pixels it saves as PNG are floats. Validators such as TFDV [4], Deequ [5], Great Expectations [6] and pandera [7] check the data, yet they never look at the code that transforms it.

Batch preparation offers something a compiler can rarely count on: the input is already on disk when the pipeline is compiled. PrepLang reads it. Where F# type providers infer types from a sample [8], PrepLang takes values from every CSV cell, every Parquet footer and every image header, and carries what it learns (column types, missing values, numeric ranges, image sizes and channels) from one statement to the next. Each contract is proven or refuted before any row is processed, and checks or operations that the input makes pointless are removed.

A proof about this month's file does not carry over to next month's, even when the layout is unchanged. For each removal, PrepLang therefore records the facts it relied on, the removal's *item certificate*, and the generated program tests them against every new batch. While they hold, the removal stands. When a batch breaks them, the removed code runs again for that batch, much as dynamic compilers fall back from optimized code [9, 10]; we call this *selective deoptimization*. A numeric column may also switch between whole and decimal numbers from one batch to the next, provided that compiling with the other type yields the same program, and the column is then read with whatever type the batch has. A batch is refused only when no run-time code can recover a fact, for instance when a numeric column arrives as text. A user compiles a `.prep` program against the batch at hand and deploys the generated program for the batches that follow.

Table 2 sets PrepLang against the closest tools. Validators see the values but check them away from the code, on every batch; static analyses follow the code but see only declared or inferred structure. We know of no other tool that decides value contracts at each point of a program from the real data and carries those decisions forward to later batches.

2. Software description

2.1. Software architecture

The compiler (`src/`, about 11,800 lines of C++17 with no external libraries) is a conventional pipeline of lexer, LL(1) parser, semantic analysis, IR, optimizer, certifier and code generator (Fig. 1). Its schema readers scan CSV files in parallel and read only the footers of Parquet files and the headers of PNG and JPEG

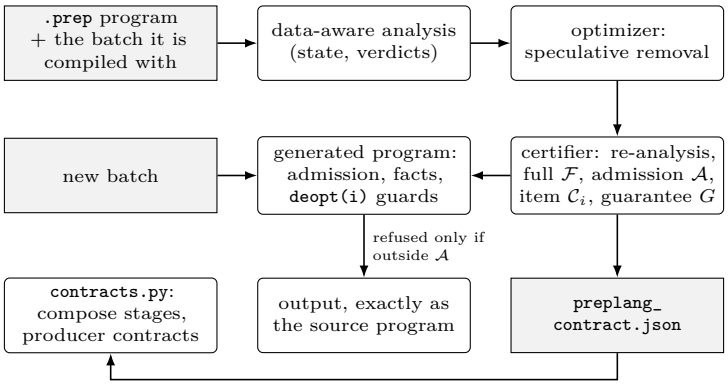


Figure 1: How PrepLang compiles once and reuses the result. The certifier re-runs the analysis with weaker input facts, reading no data; on a new batch, the generated program runs a removed instruction i only if the batch breaks its item certificate $\mathcal{C}_i$.

images. The semantic analyzer builds an abstract state of each dataset (types, missing values, value intervals, image properties) and decides every contract against it. The optimizer removes, speculatively, whatever that state shows to have no effect. The certifier then reanalyzes the program under weaker input facts, without touching the data, to find the admission certificate and one item certificate per removal. Finally, the back end emits a single Python program that admits or refuses each batch, guards every removed instruction with `deopt(i)`, and writes the certificates to `preplang_contract.json`.

`proofs/` holds 1477 lines of Lean 4 [14] that prove two results: for any sound and monotone analysis, the generated program agrees with a fresh compilation on every batch it admits; and the analysis of a core of the language is sound and monotone. Joins, groups, dates and images fall outside that core. The differential test of Section 4 covers joins and groups, while dates and images are covered only by unit tests and the examples. `tests/` contains 306 checks in 10 suites.

PrepLang can be installed with `pip` or run from the `Dockerfile`, and each release ships prebuilt compilers for Windows, Linux and macOS. A VS Code extension (`editors/`) shows the diagnostics when a file is saved. Continuous integration builds and tests the compiler on three systems (Windows with MSVC, Ubuntu and macOS with GCC), compiles every example, runs the tests under AddressSanitizer and UndefinedBehaviorSanitizer with a coverage gate, fuzzes the three parsers of untrusted input (`.prep`, CSV, Parquet footers) with libFuzzer, and checks the Lean proofs. `LANGUAGE.md` documents the syntax, the command-line options and the generated program, and `ARCHITECTURE.md` maps each phase to its source files.

2.2. Software functionalities

- **Compile-time checking.** Misspelled or dropped columns, type errors and broken contracts (`NOT_NULL`, `BETWEEN`, `UNIQUE`, `REFERENCES`, schemas, image channels and sizes, match rates between images and labels) are reported with the line, an offending row and a hint, as are joins on keys of different types. A lookup join expects each key once, while `JOIN ... MANY` keeps every match. Groups (`AGGREGATE`, with eight functions from `COUNT` to `STD`) keep their facts.
- **Data-driven optimization.** Proven checks and operations without effect are removed, unread columns are never loaded, and image pipelines stream one file at a time; rewrites that alter the output need `--approx`.
- **Tools around the compiler.** `tools/contracts.py` checks the guarantees of one compiled stage against the certificates of the next. `frontend/pandas2prep.py` translates the cleaning statements of a pandas script into PrepLang, up to the first statement it cannot express. With `--facts`, a program compiles from differentially private facts released by the data owner (`tools/release_facts.py`) without opening any input; since the generated program checks those facts on the real data, every run is still exact.

- **Inspection.** Each phase can be printed (`--tokens` through `--python`), along with an execution plan (`--explain`) and JSON diagnostics (`--json`). `PLOT` draws histograms and bar, scatter and line charts with matplotlib, marking contract bounds, and `REPORT` writes a web page comparing each column of a run with the compiler's view of it.

3. Illustrative example

```
LOAD "students.csv" AS students;
EXPECT students.age NUMBER;
EXPECT students.marks BETWEEN 0 AND 100;
EXPECT students.name NOT_NULL;
FILTER students WHERE marks >= 50;
EXPECT students.marks BETWEEN 60 AND 100;
SAVE students AS "clean.csv";
```

Compiled against the eight-row `students.csv` in `examples/tabular/`, PrepLang proves the first three contracts. The fourth stays a run-time check: after the filter, all the compiler knows is that the marks lie in $[50, 95]$. The range and not-null checks are removed speculatively, and `--certificate` lists what the range check depends on, namely marks of at least 0 and at most 100. Given a later batch that contains a mark of 104, the generated program puts the check back and stops exactly where a fresh compilation would:

```
Deoptimized line 3: EXPECT students.marks BETWEEN 0 AND 100:
  marks goes up to 104, above 100
ContractError: line 3: students.marks has values outside 0 to 100
```

A batch that stays within the facts runs with both checks removed. A batch in which `age` holds text is refused, because no generated code can stand in for analysing the column under another type.

Image pipelines get the same treatment, from file headers alone. `examples/images/linked.prep` links images with their labels, checks them, resizes them, scales the pixels to $[0, 1]$ and saves NumPy arrays. Switching the output format to PNG produces this error before a single pixel is decoded:

```
error[type]: cannot save FLOAT32 pixels as PNG:
             NORMALIZE at line 10 scaled them to [0, 1]
   --> examples/images/linked.prep:13:1
13 | SAVE_IMAGES imgs TO "processed/ml_ready/";
   = hint: image files store 8-bit pixels, so these would come out
     almost black; add CONVERT imgs TO NPY; to keep the float
     values
```

The equivalent OpenCV script runs to completion and writes images that are nearly black.

4. Impact

PrepLang is a research prototype, started in a compiler-design course, and has not yet been used outside the authors' own experiments. The evidence for it is the evaluation in `eval/`; `eval/evaluate.py` derives every number below from the raw results.

- **Correctness.** We generated 8,000 random programs over four seeds, with groups, lookup joins and `MANY` joins (1,018 programs). Compiled with and without the optimizer, each produces the same table, value for value, as an independent Python interpreter of the language that shares no code with the compiler. Run again on a second random batch, none of the 1,569 and 4,814 reuses disagrees with the interpreter. In 1,911 of those batches we flipped one numeric column between whole and decimal values: 1,341 ran or stopped as the interpreter did, 570 were refused (57 of them could not be compiled at all), and none was wrong. Along the way the test exposed three bugs: column pruning and a pandas 3 incompatibility, which together made 109 optimized runs wrong, and a statistic fill wrongly removed as a dead store (1 run). The CSV fuzzer found a fourth, an out-of-bounds read on a NUL byte. Each fix has a regression test.

- **Performance.** On a 200,000-row table the optimized program needs 0.81 s instead of 1.32 s once Python has started. Most real inputs are far smaller, and over the 156 census programs that the optimizer changes the median speedup is only 1.05× (at most 1.6×). The image pipeline is as fast as a hand-written streaming script and uses 10 MiB, against 922 MiB for a script that loads every image first.
- **Optimization.** Over 452 programs (the examples and the programs lifted from GitHub scripts), the optimizer cuts 14% of the instructions (2,157 to 1,854), the run-time checks from 43 to 8 and 2% of the columns read. It changes 156 programs, and all 452 write identical files with and without it. The lifted scripts carry no contracts, so every removed check comes from the examples or the monthly pipelines.
- **Real programs.** Of 44 data-preparation bugs reported in GitHub issues, PrepLang catches 18 at compile time, 6 of them once the requirement is written down as an `EXPECT`. The lifter translates 378 of 733 public scripts written for the Kaggle Titanic and House Prices data. Their programs match the original script on 4131 of 4158 new batches and stop or refuse on the rest; none is wrong. Among 945 traffic and bike-sharing scripts it translates 85, and these match the script on every one of 3710 later months of real data. For 469 of the 945 the lifter cannot tell which file is read, because the path is known only at run time or the read options change the table. It also leaves `!=` and `~` untranslated: on a missing value pandas keeps the row, whereas PrepLang drops it.
- **Comparison.** Five monthly pipelines on public data (NYC green taxi, Citi Bike Jersey City, and the UCI traffic, bike-sharing and Beijing air-quality sets) were each compiled on one month and run on the months after it. They match a fresh compilation on 426 of 426 months (100%) and never produce a different result. That includes the taxi and air-quality months in which a column switched between whole and decimal numbers, which version 2.0 refused. The Citi Bike and air-quality pipelines were written for this version, with bounds taken from the domain (the New York area, physical limits) rather than from the data. Each validator learned from the compiled month as its documentation recommends (Great Expectations profiles the columns, pandera and TFDV infer a schema) and was then asked whether the pipeline could run on each later month; its answers were scored against the same fresh compilation. Great Expectations and pandera reject 96% of the months, and TFDV accepts 10 months on which the program then gives a different result. These tools were not designed for this decision, so the figures measure the cost of using them for it. All 24 mistakes injected into 5 example programs are reported at compile time, while a script-style pandas program crashes on 20 and silently writes wrong output on 4.
- **Private compilation.** At $\varepsilon = 1$, compiling from released facts keeps every removal in the taxi and traffic pipelines but only 14% in the bike pipeline, where AIM [15] keeps 54%. In 74 of 184 Titanic compilations a released fact proves false, and a removal is undone on the very file that was compiled. Across 64,203 runs, no output differs from a fresh compilation.
- **Overhead.** Compilation takes a median of 125 ms, most of it spent on the extra compilations that decide which numeric columns may change type, and never more than 24 MiB of memory (65 MiB for a million CSV rows). Admitting a new batch by its certificate changes the run time by −1% to +2%, which is within measurement noise.

`eval/reproduce.sh check` (about ten minutes, offline, also in Docker) rebuilds the compiler, runs every test and regenerates these numbers from the committed results; `full` (several hours) downloads the inputs, including the GitHub scripts pinned by URL, commit and hash, and reruns every experiment, and `baselines` the validators.

5. Limitations and conclusions

Joins match on a single key column, and Python is the only target. A numeric column may change type only if no statement combines it with another column; any other change of type is refused. The mechanized proofs cover a core of the analysis that leaves out joins, groups, dates and images. Next on the list are joins on several columns and code generation for other targets.

Declaration of competing interest

The authors declare no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Use of generative AI

Generative AI tools assisted with implementation, literature search, evaluation scripts, analysis and drafting of this manuscript. The authors reviewed all text, code and results and take responsibility for them.

References

[1] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, "Hidden technical debt in machine learning systems," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2015, pp. 2503–2511.

[2] W. McKinney, "Data structures for statistical computing in Python," in *Proc. 9th Python in Science Conf. (SciPy)*, 2010, pp. 56–61.

[3] G. Bradski, "The OpenCV library," *Dr. Dobb's Journal of Software Tools*, 2000.

[4] E. Breck, N. Polyzotis, S. Roy, S. E. Whang, and M. Zinkevich, "Data validation for machine learning," in *Proc. Conf. on Machine Learning and Systems (MLSys)*, 2019.

[5] S. Schelter, D. Lange, P. Schmidt, M. Celikel, F. Biessmann, and A. Grafberger, "Automating large-scale data quality verification," *Proc. VLDB Endowment*, vol. 11, no. 12, pp. 1781–1794, 2018.

[6] Great Expectations, "Great Expectations: data quality testing," `https://greatexpectations.io`.

[7] N. Bantilan, "pandera: Statistical data validation of pandas dataframes," in *Proc. 19th Python in Science Conf. (SciPy)*, 2020, pp. 116–124.

[8] T. Petricek, G. Guerra, and D. Syme, "Types from data: Making structured data first-class citizens in F#," in *Proc. 37th ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, 2016, pp. 477–490.

[9] U. Hölzle, C. Chambers, and D. Ungar, "Debugging optimized code with dynamic deoptimization," in *Proc. ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, 1992, pp. 32–43.

[10] A. Barrière, S. Blazy, O. Flückiger, D. Pichardie, and J. Vitek, "Formally verified speculation and deoptimization in a JIT compiler," *Proc. ACM Program. Lang.*, vol. 5, no. POPL, pp. 1–26, 2021.

[11] Y. Y. Zhuang and M.-Y. Lu, "Enabling type checking on columns in data frame libraries by abstract interpretation," *IEEE Access*, vol. 10, 2022.

[12] O. Gerstl, F. Sihler, and M. Tichy, "Inferring the shape of data frames in R programs using abstract interpretation," arXiv:2607.03889, 2026.

[13] V. Mirji, "Shift schema drift left: Policy-aware compile-time contracts for typed JVM and Spark pipelines," arXiv:2604.16986, 2026.

[14] L. de Moura and S. Ullrich, "The Lean 4 theorem prover and programming language," in *Proc. 28th Int. Conf. on Automated Deduction (CADE)*, LNCS 12699, 2021, pp. 625–635.

[15] R. McKenna, B. Mullins, D. Sheldon, and G. Miklau, "AIM: An adaptive and iterative mechanism for differentially private synthetic data," *Proc. VLDB Endowment*, vol. 15, no. 11, pp. 2599–2612, 2022.